AAC-INV-001 · SECTION 06

Platform Asset Value

This chapter makes the asset case in terms a diligence team can verify: what exists, what it would take to recreate, what risks it retires, and where the value concentrates — including the key-person dimension.

No revenue figures or valuations are asserted here; those belong to management's financial materials. What this chapter provides is the **verifiable substance** any such figures would rest on.

§ What exists, measurably

All numbers below are derived directly from the repositories and can be re-counted at any time:

METRIC	VALUE	WHERE TO VERIFY
Database migrations (ordered, reproducible)	**503**	`silkskyair-api/supabase/migrations/`
Dedicated database schemas	**16**	schema creation migrations
Tables defined over the platform's life	**~160** (140+ in active service)	migrations
Seed files for full-environment reproduction	**30**	`silkskyair-api/supabase/seeds/`
Supabase edge functions	**3**	`silkskyair-api/supabase/functions/`
Event-driven n8n workflows	**20** across 7 categories	`silkskyair-workflows/workflows/`
Back-office modules (permission-gated)	**26**	`silkskyair-manager/lib/modules/registry.ts`
Repositories in the estate	**19**	the `andaman-aerodrome` GitHub organization
Published shared libraries	**3** (`ui`, `reporting`, `skystories`)	GitHub Packages
Deployable applications/services	**8** (manager, member, partner, account, www, cms, workflows, api)	repo estate
Languages	EN + TH live; RU + ZH provisioned	`i18n` schema and translation tables
Booking lifecycle event types	**40+**	`booking_event_types`

§ Replacement-cost reasoning

The honest way to price a software asset without invented numbers is to enumerate what a buyer would otherwise have to build. Recreating this platform requires, at minimum:

1. **An aviation network domain model** — airfields, route graph, fleet, maintenance, missions, crews, disruptions — plus a **scheduling and availability engine** with caching and nightly recomputation. This is the rarest part: it requires aviation domain knowledge, not just engineering hours.
2. **A complete commerce stack** — catalog, component-based dynamic pricing, event-sourced bookings, two payment-provider integrations with reconciliation, refunds, commissions with Thai VAT/withholding-tax mechanics, customer identity, promotions/coupons/campaigns.

3. **Five audience-specific applications** plus a headless CMS and a workflow engine, integrated end-to-end (email, CRM, payments, link/QR services).
4. **The operational hardening that only production exposure produces** — the 503 migrations are a fossil record of real-world edge cases already paid for: amendment workflows, settlement disputes, cache invalidation, locale gaps, partner review cycles (documented in this repository's `plans/` and `manuals/`).

Item 4 is the moat. A competitor can hire a team and copy a feature list; it cannot shortcut the iteration history with live customers, partners, regulators and payment providers.

---

§ Risk already retired

Investors price risk; this platform has already retired several classes of it:

- **Technical risk** — the system runs in production with staging/production environments, CI/CD, E2E test orchestration and documented releases.
- **Integration risk** — payments (Omise, SCB), CRM (Zoho), CMS (Strapi), automation (n8n) are live integrations, not roadmap promises.
- **Compliance posture** — event-sourced bookings and settlements give tamper-evident audit trails; access control is enforced in the database (RLS), not just in app code; no card data is stored (tokenized payments).
- **Market-expansion risk** — multilingual, multi-timezone and multi-currency-ready by schema design; adding a market is content and configuration work.
- **Key-platform-vendor risk** — the core is standard PostgreSQL and open-source-centric tooling (Strapi, n8n); managed services are conveniences, not lock-ins.

---

§ Strategic optionality

Beyond its operating value, the platform carries **option value** that conventional booking software does not:

1. **White-label licensing** — the separation model turns the commercial layer into a sellable product for other operators on the network.
2. **Clean divisibility** — network and commercial layers can be held, financed or sold as distinct assets along boundaries that already exist in the schema, privilege model and repository structure.
3. **Activity expansion** — charters, transfers, medevac and ferry operations are existing mission types of the same engine; new aviation businesses launch on infrastructure that is already amortized.
4. **Network effects** — each new helipad/heliport node increases the value of every existing node and of every operator on the platform.

§ The Platform Architect — key-person value

The platform was conceived, designed and delivered under a **single platform architect**, and that fact is visible in the artifact itself:

- **Coherence.** One design philosophy runs through 19 repositories: the same separation of network and commerce, the same module/privilege naming, the same event-driven patterns, the same documentation discipline. Systems assembled by rotating teams do not look like this; the absence of architectural drift is itself evidence of concentrated authorship.
- **Velocity.** The measurable output above — 503 migrations, 8 deployed applications/services, 3 published libraries, 20 production workflows, multilingual content systems — was produced at a pace and consistency that a hired multi-vendor team would struggle to match at any budget.
- **Institutional knowledge.** The architect holds the full map: why each boundary sits where it does, which constraints are aviation-domain requirements versus implementation choices, and how the white-label separation is meant to unfold. This knowledge is the difference between a platform that compounds and one that calcifies after handover.
- **Continuity plan, not bus-factor excuse.** The risk that key-person value usually implies is actively mitigated here: reproducible environments, ordered migrations, seed data, staff manuals and weekly engineering reports mean the platform is *transferable* — but its **strategic direction and extension velocity remain coupled to its architect**, which is precisely why retaining that role is part of the asset.

For an investor, the conclusion is straightforward: the platform and its architect are complementary assets. The codebase de-risks continuity; the architect de-risks evolution — the expansion into new nodes, new operators and new aviation activities that this investment thesis is about.